CASE — YEZZZ

Hero

Yezzz Mobile operator · Funnel & conversion redesign · iOS, Android · 2026

📸 [IMAGE: Hero — full-width split mockup showing the before/after of the Yezzz landing screen inside the Silpo app. Bold metric overlay: +69.1% conversion rate.]

Role — Product Designer

TL;DR

Yezzz is a mobile operator embedded in Ukraine's largest supermarket app — users get free mobile service simply by doing their regular grocery shopping. The offer was genuinely strong. The problem was that almost no one understood it well enough to act on it. I led the redesign of the full activation experience — from first impression through post-activation — identifying where comprehension broke down, why users lost trust, and what it would take to make a compelling product feel as compelling as it actually was.

The Problem

Yezzz had a unique value proposition: mobile coverage you effectively pay for through your grocery shopping. The user base was already there — millions of loyal Silpo app users. The problem wasn't demand. It was comprehension and trust.

A 2.35% conversion rate on a strong offer is a signal that the barrier is experiential, not motivational. And the data confirmed it: **87% of users dropped off at the very first screen** — before reaching the activation flow at all.

📸 [IMAGE: Funnel visualization — users entering at the Yezzz screen (138,482) narrowing to activations (3,251). The 87% drop at step one highlighted clearly.]

Research & Discovery

We ran 15 in-depth interviews across 3 user groups — people at different stages of interaction with Yezzz. Each session lasted ~20 minutes and focused on three areas: the

activation experience, service comprehension, and what drives or blocks trust when switching mobile operators.

Before activation — Users who didn't understand the mechanic upfront simply lost interest. The offer read like a one-time promotion, not an ongoing service model.

During activation — The flow confused users. Many couldn't tell if they were completing steps correctly. The number portability process felt opaque and high-stakes, even though users reported being less afraid of switching operators than assumed.

After activation — Users couldn't track their tariff status. They didn't know which billing period was active, whether they'd met the spending condition, or what would happen if they fell short.

📸 [IMAGE: Research synthesis — three columns labeled Before / During / After activation, each populated with key findings and representative user quotes]

📸 [IMAGE: Key quotes panel — 4 quotes from research participants displayed as large pull-quotes. e.g. "Did I meet the condition in February? Will I get free service in March? How am I supposed to know?" — Tetiana]

The consistent theme across all three groups: the product was working. The communication around it wasn't.

Product Direction & Strategy

Research surfaced ~15 hypotheses. We scored each using the ICE framework to separate high-leverage interventions from noise. This produced a clear priority stack.

The highest-priority issues clustered around two problems:

Comprehension before activation — Users needed to understand what they were signing up for before being asked to act. Any friction introduced before comprehension was established would cause drop-off, and it did.

Visibility after activation — Users who successfully activated had no reliable way to track their status. This eroded trust and reduced the likelihood of re-engagement the following month.

📸 [IMAGE: ICE scoring matrix — hypotheses plotted on a grid, top-scoring ones highlighted with brief rationale labels]

Deep Dive 01 — Transparent Conditions Before Activation

The Problem

Users arriving at the Yezzz screen didn't know what the service required of them. The spending condition (₴2,000/month) was only revealed after the first purchase. Without that information upfront, the offer felt ambiguous — and ambiguous offers get ignored.

The Decision

I restructured the main screen to lead with the mechanic, not the marketing. The value proposition — what you get, what it costs, how the monthly cycle works — was explained in plain language before any action was requested. Benefits were listed clearly. The ₴200 fallback tariff was shown alongside the free tier so users understood the full picture.

The goal: by the time a user reaches the activation CTA, they should have no unanswered questions about what they're signing up for.

📸 [IMAGE: Before/after — original Yezzz landing screen (vague, marketing-forward) vs. redesigned screen (mechanic explained, conditions visible, benefits listed)]

Deep Dive 02 — Post-Activation Status Visibility

The Problem

Activated users had no clear way to track their tariff status. They couldn't see their remaining minutes or data, identify which billing cycle was active, or know whether they were on track to meet the monthly spending condition for free service.

This wasn't just a UX gap — it was a trust problem. Users who can't tell if a product is working for them assume it isn't.

The Decision

I added visual usage progress indicators for minutes and data directly to the account screen — a quick-read status that answered the most common post-activation questions without requiring any navigation. Billing cycle dates and free package conditions were surfaced inline, with clear language about what the current cycle required and when it would reset.

I also added a dedicated FAQ section addressing the most common questions about tariff conditions, the spending mechanic, and billing rules. Confusion had been building silently — this gave users a self-serve path to answers without needing to contact support.

📸 [IMAGE: Account screen before — sparse, no usage progress, no cycle information visible]

📸 [IMAGE: Account screen after — usage progress bars for minutes and data, billing cycle dates, package condition status, FAQ entry point]

Onboarding: Before and After Activation

Two onboarding flows were added at different points in the funnel, each solving a distinct problem.

Pre-activation onboarding — New visitors to the Yezzz section had no context for what the service was. Without a clear introduction, there was no basis for a decision. I added a brief onboarding flow that explains what Yezzz is and what it offers before asking for any commitment.

Post-activation onboarding — Newly activated users didn't know where to find their package status or what they could do inside the Yezzz section. A guided walkthrough of the account screen — package status, balance, billing cycle — was added immediately after activation.

📸 [IMAGE: Pre-activation onboarding — 3-screen sequence explaining Yezzz before the activation CTA appears]

📸 [IMAGE: Post-activation onboarding — guided overlay on the account screen highlighting key areas: balance, cycle status, FAQ]

Validation

Updated designs were tested with 43 participants: 13 in moderated Figma sessions for behavioral depth, and 30 in unmoderated tests via Maze and Lyssna for broader coverage.

Testing covered three stages: first impression and offer comprehension, activation flow navigation, and post-activation status understanding.

Users understood the tariff conditions significantly better than in the original flow and moved through activation with noticeably more confidence. The moderated sessions surfaced two edge cases that were addressed before shipping.

📸 [IMAGE: Validation setup — split showing moderated session (Figma prototype) and unmoderated test (Maze results overview). Key task completion rates annotated.]

Reflection

The problem wasn't the product — it was the gap between what the product offered and what users understood. Yezzz had a genuinely strong value proposition. The work wasn't about changing it, but about making it visible and believable at the right moments.

Qualitative research unlocked things analytics couldn't. The funnel showed where users dropped off. Interviews explained why. The insight that users didn't know they'd already met the conditions for free service — invisible eligibility — came entirely from conversations, not from data.

Constraints shaped better decisions. The MNP flow couldn't be redesigned due to legal requirements. The main app screen had limited real estate. Working within those boundaries pushed toward communication and framing solutions rather than structural ones — and those turned out to be exactly what was needed.

CASE — PHLEX

Hero

Phlex Swimming performance tracker · Product design · iOS, Android, Web · 2022–2025

📸 [IMAGE: Hero — full-width phone mockup showing the main training dashboard mid-session. Live metrics visible: pace, stroke rate, heart rate, SWOLF. Dark water-inspired background — deep navy or near-black with subtle motion. Bold, athletic feel.]

Role — Lead Product Designer

TL;DR

Phlex is a swimming performance tracker that connects with hardware wearables to capture pace, stroke rate and heart rate in real time — then turns that data into something swimmers can act on. I led product design across three years of continuous iteration: from the MVP to gamification, social competition, and a coach-facing web dashboard. The core challenge throughout was making metrics feel motivating and accessible — for both a casual lap swimmer and a professional athlete.

The Problem

Swimmers are chronically underserved by performance tracking tools. Running and cycling have mature ecosystems that make data intuitive and motivating. Swimming has almost

nothing that works well — most tools either require manual lap counting, fail to sync reliably with hardware, or present raw data that means nothing poolside.

Phlex had the hardware advantage. The design challenge: that data is worthless if swimmers can't read it quickly, understand what it means, and feel motivated to return tomorrow.

A second challenge: Phlex had to serve two fundamentally different users inside one product: swimmers tracking personal progress, and coaches managing a roster of athletes without the product feeling split or bloated.

📸 [IMAGE: Problem framing — two-column visual. Left: "What data exists" (raw metrics). Right: "What swimmers actually need" (insight, progress, motivation).]

Research & Discovery

I ran structured user interviews at multiple stages — at launch, at 12 months, and beyond. Continuous interviews were the primary input for every major roadmap decision.

Three findings shaped the product at each stage:

Metrics without context intimidate instead of motivate. Casual swimmers saw their score and felt confused. Professional swimmers wanted granular data but needed it readable in seconds poolside, between sets, wet hands.

Solo tracking loses users after the first few weeks. Without external accountability, motivation fades once initial novelty wears off. Swimmers who trained with clubs retained far better — not because of coaching, but because of social structure.

Coaches were managing athlete progress in spreadsheets. No visibility into training data between sessions. Screenshots from athletes, manual trend tracking. A clear product opportunity with obvious professional frustration behind it.

📸 [IMAGE: Research timeline — horizontal line showing interview rounds at MVP / 6mo / 12mo / 2yr, with the key insight from each round annotated]

📸 [IMAGE: Key quotes — 3 swimmer quotes and 2 coach quotes as large typographic pull-quotes]

Deep Dive 01 — Onboarding: Making the Product's Depth Visible

The Problem

Phlex launched without formal onboarding. Early on, that was fine — the product was small and the core tracking loop was discoverable. Two years in, the product had grown considerably: gamification, goal tracking, competition features, coach tools. Features that took months to build were invisible to new users who'd never been shown they existed.

New users would track a few sessions and churn — never reaching the features that would have kept them engaged. Retention data showed a consistent drop-off at the end of week one.

The Decision

I designed a focused onboarding flow: a short sequence of screens shown on first launch that introduced Phlex's key features directly, before the user reached the main interface.

Each screen highlighted one feature category (performance tracking, levels and progress, group competition, coach tools) with a clear visual and a single-sentence explanation of why it mattered. Short, skippable, purposeful.

Impact

Users accessing at least 3 distinct feature areas in their first 14 days increased from **~29% to ~61%.**

Deep Dive 02 — Gamification: A Points System Tied to Real Performance

The Problem

Solo performance tracking has a well-documented drop-off curve. The first few weeks are high-engagement — everything is new, progress is visible. After that, without a reason to return beyond habit, users disengage. Day-30 retention was where Phlex was losing people.

The Decision

I designed a points-and-levels system where every workout generates a score — calculated from the session's actual metrics weighted against the user's stated training goals. Those points accumulate across workouts and advance the user through a progression of levels. The system reflects genuine performance, not just frequency.

This was important for credibility. Swimmers know their times. A gamification system that rewarded attendance without recognizing performance would have been dismissed immediately by serious users. Tying points to real metrics made the levels feel earned.

Impact

Day-30 retention increased after gamification launched. Users actively progressing through levels showed **higher** session frequency than users who hadn't engaged with the system.

Deep Dive 03 — Group Competition: Adding the Social Layer

The Problem

Gamification improved retention significantly, but it relied entirely on intrinsic motivation — personal goals, personal records, personal levels. For a large segment of users, that wasn't enough to sustain long-term engagement.

Research had shown that swimmers who trained with clubs retained far better than solo users. The social structure — knowing others were tracking the same metrics, competing for the same goals — was the differentiating factor.

The Decision

I designed Leagues: time-limited group competitions where swimmers competed on a shared leaderboard within a defined challenge window (typically 4 weeks). Leagues could be open (any Phlex user) or private (invite-only for clubs or friend groups).

Sponsored prizes changed the dynamic significantly. Phlex partnered with swim brands to offer gear and equipment for League winners. This drove external recruitment — users invited non-Phlex swimmers to join a League, which became one of the top acquisition channels post-launch.

📸 [IMAGE:

Impact

Leagues became the top acquisition channel within 2 months of launch. MAU grew in the quarter following release. Sponsored Leagues generated **3.1× higher** invite rates than non-sponsored ones.

Impact

- Day-30 retention after gamification launch: **+14%**
- Session frequency (level-active users vs. not): **2.3×**

- Session frequency during active League: **1.8×**

📸 [IMAGE: Impact stat block — 6 key metrics in a clean two-row grid.]

Reflection

Three years on one product clarified something about the relationship between trust and features. Every motivational layer — points, levels, competition — only worked because the underlying tracking data was reliable. Swimmers are precise people who know their times. Features built on top of that data either confirm trust or destroy it.

The evolution from individual gamification to group competition also validated a sequencing principle: solo progress metrics sustain early engagement; social accountability sustains long-term behavior. The two systems weren't alternatives — they were stages. Gamification built the performance baseline. Competition made that baseline meaningful in relation to others.

TUTORLY

Hero

Tutorly Online learning platform · End-to-end product design · 2023

📸 [IMAGE: Full-width hero

Role — Lead Product Designer

TL;DR

Tutorly needed a platform that worked for two fundamentally different people — students who want to learn, and tutors who want to build a business. I designed the entire product architecture, both role-specific experiences, and a scalable component system that supported everything from video lessons to earnings tracking — without bloating the interface.

The Problem

Most online learning platforms are built for one side of the market. Tools designed for tutors are clunky for students. Tools optimized for students give tutors almost no control over their own business.

Tutorly needed to serve both — not with a one-size-fits-all interface, but with two genuinely separate, purpose-built experiences inside the same product.

The core tension: complexity for tutors had to stay invisible to students. Every feature added for one role was a potential source of confusion for the other.

📸 [IMAGE: Early audit/whiteboard mapping the two user journeys side by side — key decision points and divergences highlighted]

Research & Discovery

I ran discovery interviews with 12 users — 6 current tutors and 6 students across different subject areas and age groups.

Three patterns became the design foundation:

Students don't want to manage — they want to progress. Every extra click between "open the app" and "start the lesson" caused drop-off. Progress visibility wasn't a nice-to-have — it was directly tied to whether students came back.

Tutors think like business owners, not teachers. Their first questions were about earnings, scheduling, and client management — not pedagogy. The product needed to support that mindset from day one.

Onboarding drop-off is where most platforms fail. Tutors who couldn't build and publish a course in their first session never returned. Students who couldn't book a lesson within 5 minutes of signing up never converted.

📸 [IMAGE: Research synthesis board — two columns of insights labeled "Student Jobs to Be Done" and "Tutor Jobs to Be Done" with key quotes]

Product Direction & Strategy

I made the early call to treat students and tutors as two separate products sharing one infrastructure — not one product with role toggles.

This meant:

- Separate onboarding flows tuned to each role's first-use goal
- Role-specific dashboards, not a universal home screen

- Shared design system under the hood, so engineering could build once

I deliberately deprioritized the social layer (reviews, ratings, Q&A) for v1. It added complexity without addressing the core activation problem. Instead, I focused the first release on three things: discovery, booking, and progress — the minimum loop to make Tutorly genuinely useful on day one.

📸 [IMAGE: Prioritization matrix / product roadmap diagram showing v1 scope decisions — what was in, what was deferred, and why]

Deep Dive 01 — Dual Onboarding

Problem The initial onboarding was a single linear flow that asked both students and tutors to complete the same steps before they could do anything meaningful. Tutors were answering questions about their learning goals. Students were being asked to set up payment before they'd seen a single tutor profile. The result: high drop-off from both sides before activation.

Hypothesis Students and tutors have completely different definitions of "done". A tutor feels activated when their profile is live and bookable, when they feel like they're open for business. A student feels activated when they've found a tutor and booked a lesson. The onboarding flow had to reflect that difference.

Decision

I split onboarding into two purpose-built paths, each ending at the user's first real action, not at email confirmation. For tutors: **bio setup → availability → profile live.** The finish line was a shareable profile URL. Seeing their own profile, bookable and real, was the activation moment.

For students: **subject or goal input → tutor browse → profile view → book.** The flow was designed to reach the first tutor profile within 3 taps. Everything else as account details, payment setup happened after the student had already chosen someone.

I also stripped both paths to the minimum viable steps. Every field that wasn't essential to the first session was deferred to a post-activation profile prompt.

📸 [IMAGE: Side-by-side onboarding flows — tutor path (3 screens) and student path (3 screens) with the divergence point annotated]

Deep Dive 02 — The Tutor Dashboard

Problem Tutors needed to manage schedules, monitor earnings, track students, assign homework, and run video sessions — all without the interface becoming a second job.

Hypothesis A single-screen hub showing today's lessons, pending tasks, and current earnings balance would reduce cognitive overhead. Tutors needed a "morning check-in" view, not a full admin panel every time they opened the app.

Decision I designed the dashboard around time, not around feature categories. The top section always shows the next lesson. Earnings and pending actions sit below as secondary context. Deep management flows (course editor, student progress, withdrawals) live one tap deeper.

📸 [IMAGE: Tutor dashboard full-screen — annotated with the 3 information zones: next session, quick metrics, action queue]

📸 [IMAGE: Earnings section close-up — balance, pending, and withdrawal flow showing the progressive disclosure pattern]

Design Decisions & Trade-offs

Shared component system vs. divergent experiences I built one design system underneath two distinct experiences. This let engineering move faster without creating UX inconsistency. The trade-off was more upfront design work — I had to think about every component in both student and tutor contexts before finalizing anything.

Video lessons in v1 The team wanted to build a full video conferencing layer natively. I pushed back. Integrating a proven third-party solution (Zoom/Daily.co) for v1 let us focus design effort on the surrounding context — pre-lesson preparation, in-lesson materials, post-lesson follow-up — which was where the real UX differentiation lived.

Homework assignment flow First iteration required tutors to build homework from scratch every time. After testing, I introduced homework templates. Reuse rate in beta: ~74%.

📸 [IMAGE: Component system overview — token grid showing how shared components adapt to both roles with different visual weight]

Impact

- Onboarding completion: **+40%** vs. initial prototype
- Tutor homework template adoption: **~74%** reuse rate
- Design system: **50+ components** across student and tutor surfaces

📸 [IMAGE: Impact metrics displayed as a clean stat block — 4 key numbers with labels]

Reflection

Designing for two user types inside one product isn't about compromise — it's about identifying where the experiences can share infrastructure and where they must diverge completely. The temptation is to find a "universal" solution. The better answer is usually to build two simple things instead of one complex one.

Progress visibility didn't just improve retention. It changed how students talked about the product — from "I use Tutorly" to "I'm working toward my certificate." Framing the product around progress rather than features was the most impactful shift of the whole project.

FRACTL

Hero

Fractl Low-code / no-code development platform · Product design · 2023

📸 [IMAGE: Hero

Role — Lead Product Designer

TL;DR

Fractl is a platform that lets users build complex business applications through data modeling and visual logic without writing infrastructure code. The core design challenge: make an inherently complex, developer-grade tool feel accessible to non-technical builders, without patronizing the developers who needed its full power. I led the product design from zero, defining the interface model, designing three distinct user modes, and building the scalable design system that supported the entire platform.

The Problem

Building business applications typically requires weeks of backend setup before any visible progress. Database schemas, API connections, business logic - all of it must be configured

before a single screen can function. For non-technical users, this is a hard wall. Even experienced developers find it repetitive and time-consuming.

Research & Discovery

I mapped two distinct user personas based on stakeholder interviews and comparable tools analysis:

Professional developers — want power, speed, and flexibility. They'll use visual tools if it's faster than writing boilerplate, but they need escape hatches to code when required.

No-code builders — typically product managers or business analysts. Comfortable with logic and data concepts, uncomfortable with code syntax. They need the visual model to be the primary interface, not a secondary companion to code.

The critical insight: all personas need to coexist in the same platform. A team might have one developer and three business analysts. The platform had to feel native to all of them.

📸 [IMAGE: persona cards

Product Direction & Strategy

I proposed the "three modes, one model" architecture: a single underlying data model that could be viewed and edited through three different interfaces — visual canvas, low-code editor, and AI prompt — all staying in sync.

This was the highest-risk architectural decision in the project. Keeping three editing surfaces synchronized in real-time required significant engineering investment. I worked directly with the lead engineer to define where the sync boundaries were, what state could drift, and what had to be exactly consistent at all times.

The strategic payoff: users could switch modes mid-task without losing context. A developer could sketch the data model visually, drop into code to handle edge cases, and hand off to a no-code builder — all within the same session.

📸 [IMAGE:

Reflection

The hardest problem in building tools for multiple technical levels isn't the UI — it's the model. If the underlying information architecture tries to serve everyone simultaneously, it ends up serving no one well. "Three modes, one model" worked because the model itself was clean enough to represent faithfully at different abstraction levels.

CASE 05 — OLAS

Hero

OLAS Maritime safety mobile application · End-to-end product design · iOS, Android · 2022–2023

📸 [IMAGE: Hero

Role — Lead UI/UX Designer

TL;DR

OLAS is a maritime safety system connecting wearable trackers and engine-integrated devices to a mobile app that monitors everyone on board in real time. If someone falls overboard, the app alerts the captain in seconds — and for solo sailors, automatically contacts rescue services. I designed the entire mobile experience from scratch, optimizing every interface decision for emergency conditions: sun glare, motion, wet screens, stress.

The Problem

Falling overboard is one of the leading causes of death in recreational boating. Most incidents happen in daylight, in calm conditions, within reach of help — but without a fast enough alert mechanism, the window to respond closes in minutes.

Existing safety tools were either passive (life jackets) or required the victim to activate them. OLAS's hardware closed that gap — but it needed software that could perform reliably under the exact conditions where everything else fails: an emergency.

The design challenge was unusually high-stakes. An interface that's confusing under normal conditions is annoying. An interface that's confusing during a man-overboard event can be fatal.

Research & Discovery

I conducted research across three dimensions: user scenarios, hardware behavior, and environmental constraints.

User scenarios — I mapped four primary scenarios: multi-person recreational sailing, family sailing with children and pets, solo sailing (highest risk), and commercial/charter use. Each scenario had different alert logic requirements and different primary users.

Hardware behavior — I worked directly with the hardware team to understand the signal characteristics of each transmitter type (wristband, tag, engine unit), their detection ranges, and their failure modes. Understanding hardware constraints was essential — the UI had to accurately represent device state, including degraded states.

📸 [IMAGE: Environmental constraint diagram — 4 maritime conditions with the specific UI implication of each one illustrated]

Product Direction & Strategy

I defined three distinct interface states that had to be designed with equal rigor:

Normal state — Real-time monitoring view. All devices visible, all crew accounted for. This is where users spend 99% of their time. It had to be calm, informative, and low-cognitive-load.

Alert state — Man-overboard detected. This is the state where the design had to perform perfectly under maximum stress. Every other design concern was subordinate to: can a panicked captain confirm or dismiss this alert correctly on the first try?

Setup state — Device pairing, network configuration, crew assignment. Only used at dock, before departure. This could be more complex — but still needed to be reliable, because errors made during setup would surface as false alerts during use.

I made the deliberate decision to design the alert state first, then design the normal state around the constraints the alert established.

📸 [IMAGE:]

Reflection

Designing for emergencies exposed a principle I now apply everywhere: the measure of a good interface isn't how it performs when everything is fine. It's how it performs when nothing is.

Every design constraint imposed by the maritime environment was also a general UX principle pushed to its logical extreme: high contrast, large touch targets, clear single-action states, no unnecessary complexity. Working under these constraints produced a cleaner, more intentional design than I would have reached without them.

CASE — CARE YOU

Hero

Care You Healthcare platform · End-to-end product design · iOS, Android, Web · 2022

📸 [IMAGE: Hero — full-width mockup

Role — UI/UX Designer

TL;DR

Care You connects patients, carers, clinicians, and insurers in one platform — reducing hospital readmissions by improving coordination during the highest-risk period: post-discharge home care. I designed the entire multi-role mobile experience and admin panel, and built a flexible design system that healthcare organizations could brand without custom development.

The Problem

Hospital readmissions within 30 days of discharge are one of the most expensive and preventable problems in healthcare. Most re-admissions aren't medical failures — they're coordination failures. Patients don't know what to do at home. Carers don't have the right information. Clinicians can't easily monitor risk between appointments.

Existing tools either serve only clinicians (complex, not patient-friendly) or only patients (disconnected from clinical workflows). Nothing connected all four roles — provider, insurer, patient, and carer — in a shared ecosystem.

Product Direction & Strategy

I defined the core product bet early: **Care You's value lives in coordination, not features.**

The platform's job wasn't to add new medical capabilities — it was to make the coordination between existing roles faster, clearer, and less error-prone. That reframing influenced every prioritization decision.

For the v1 scope, I pushed to ship:

1. Care network creation (connecting patient + carer + clinician in one shared context)
2. Task assignment and tracking
3. Risk assessment questionnaires
4. Educational content library

I deprioritized real-time monitoring hardware integration for v1 — the software coordination layer had to prove value before adding device complexity.

For healthcare organizations, the business requirement was clear: branding customization without custom dev work. I designed a theming system that allowed organizations to apply brand colors, logos, and tone of voice across all four role experiences with a single configuration layer.